

EEE3097/8/9S Micromouse 2026: Final Demonstration Instructions

Autonomous Maze Solving & Speed Run (Submission 4 - 25%)

1. Objective

Design and implement the complete autonomous intelligence for your Micromouse. The robot must explore a 4x6 grid maze to discover its wall layout, build a topological map of the grid, calculate the optimal shortest path back to the starting cell (or to the target cell), and execute a high-speed “solving run” without colliding with any walls.

Specific Learning Objectives:

- Interfacing with three VL53L0X Time-of-Flight (ToF) sensors and calibrating wall detection thresholds.
 - Fusing high-resolution wheel encoders with the gyroscope yaw rate to track coordinate state (x, y) and heading orientation (North, East, South, West).
 - Implementing grid-based exploration state machines (e.g. Floodfill or Depth-First Search).
 - Implementing shortest-path planning solvers (e.g., A*, Dijkstra, or BFS) to calculate optimal routing.
 - Tuning velocity profiles (accelerations, corner deceleration limits) to transition smoothly between straight segments and cornering.
-

2. Step-by-Step Implementation Guide

Step 1: Exploration & Mapping Run

- Place your mouse at the starting cell $(0, 0)$.
- The mouse must autonomously explore the 4x6 maze. As it enters each cell, it must read its ToF sensors, classify wall presence, and update its internal map array.
- Implement active wall-centering using side ToF measurements to dynamically correct steering drift.

Step 2: Path Solving

- Once exploration is completed, your algorithm must import the mapped grid matrix.
- Calculate the shortest path from the starting cell $(0, 0)$ to the target cell.
- Return the mouse autonomously to $(0, 0)$, re-align heading orientation, and halt to prepare for the speed sprint.

Step 3: High-Speed Speed Run

- Load the calculated shortest path array.
 - Execute a high-speed sprint directly to the target cell.
 - Your velocity planner must merge consecutive straight cells into a single acceleration-cruise-deceleration profile (rather than stopping at every cell boundary).
 - The run is successfully complete when the mouse stops autonomously and safely within the target cell.
-

3. Deliverables (Gradescope Submission)

To package your final submission, run the following command from your repository root:

```
python tools/package_submission.py --task final_demo --src workspace/final_task/
```

This script will perform dynamic syntax and formatting checks and generate a single `submission_final_demo.zip` in your project root. Upload this ZIP directly to Gradescope.

The ZIP package will automatically contain:

1. **Your Solving Code:** * **Python track:** All `.py` scripts and custom libraries recursively from your workspace. * **Simulink track:** Your model file (`.slx`) and the generated C code-generation directory (`*_ert_rtw/`).
2. **Physical Telemetry Log** (`run_log.jsonl`): * The telemetry log file of your best physical exploration and speed runs.
3. **Physical Run Video** (`run_video.mp4`): * A single, unedited video demonstrating the physical run.

4. Video Requirements & Academic Honesty Declaration

To verify that your physical run is authentic, the video must strictly adhere to the following sequence:

1. **Student Card Close-up:** The video **MUST start with a clear, readable close-up of your physical Student Card** for at least 3 seconds (declaring this is your own work).
2. **Setup:** Show the mouse positioned at the starting cell.
3. **Traversals:** Capture the mapping run, the return-to-start orientation reset, and the final high-speed run to the target cell without cuts.

5. Code & Log Correlation Verification (Anti-Cheat Check)

- **Hardware ID Check:** The `"uid"` field represents your microcontroller's unique device ID. While this is not registered in advance, the course convenors check the submitted logs for duplicate UUIDs. Submitting logs with identical UUIDs under different student accounts indicates shared files/hardware runs and will trigger a plagiarism audit.
 - **Code Match Check:** The autograder compiles and computes an FNV-1a checksum hash of your submitted code and matches it against the `"hash"` field in your telemetry header. **Mismatches will result in an immediate submission rejection.**
-

6. Evaluation Criteria

A. Co-Simulation Speed & Accuracy (Automated Check) The autograder will execute your solving code in the visual simulation testbed across multiple randomized maze layouts. * **Exploration Rate:** The mouse must correctly map at least 80% of the walls. * **Speed Run Execution:** The mouse must execute the run without collisions. Your grade is directly proportional to the traversal speed.

B. Physical Validation (Live Demo Check) You will present your mouse for a live physical evaluation run in the lab maze. The mouse must successfully map, return, and solve the layout under physical timing limits.